HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	**: Kim Ngọc Bách**
Họ và tên sinh viên	**: Đỗ Bình Minh**
Ngày sinh	**: 24/12/2005**
Số điện thoại	**: 0329279863**
Mã sinh viên	**: B23DCCN542**
Lớp	**: D23CQCN10-B**
Nhóm lớp	**: 11**

Hà Nội – 2026

Contents

1. Tóm tắt các công việc đã thực hiện trong tuần. 3
2. Chi tiết các công việc đa thực hiện trong tuần 3
2.1. Trang thông tin cá nhân (ứng viên) 3
2.1.1 Xử lý bên phía Frontend 3
2.1.2 Xử lý bên phía Backend 8
2.1.3. Kết quả 11
2.2. Trang thông tin công ty (nhà tuyển dụng) 12
2.2.1. Xử lý phía Frontend 12
2.2.2 Xử lý bên phía Backend 15
2.2.3. Kết quả 19
3. Kết quả đạt được 20
4. Khó khăn gặp phải 20
5. Hướng xử lý 21
6. Kế hoạch tuần tiếp theo 21

1. Tóm tắt các công việc đã thực hiện trong tuần.

Trong tuần trước em đã hoàn thành tính năng đăng nhập, đăng ký cho nhà tuyển dụng và ứng viên. Tuần này em sẽ hoàn thiện các chức năng liên quan đến quản lý thông tin cá nhân ứng viên và thông tin công ty. Cụ thể, em đã xây dựng giao diện hiển thị và cập nhật dữ liệu hồ sơ, triển khai validate dữ liệu ở phía frontend và xử lý upload ảnh (avatar, logo). Đồng thời, em kết nối frontend với backend thông qua các API để lấy và cập nhật dữ liệu, đảm bảo dữ liệu được lưu trữ chính xác trên MongoDB.

Kết quả đạt được là hệ thống đã hoàn thiện luồng hiển thị và cập nhật thông tin công ty, người dùng, tạo nền tảng để phát triển các chức năng quản lý công việc trong tuần tiếp theo.

2. Chi tiết các công việc đa thực hiện trong tuần

2.1. Trang thông tin cá nhân (ứng viên)

2.1.1 Xử lý bên phía Frontend

- Giới thiệu thư viện FilePond:

FilePond là thư viện hỗ trợ **upload file trên giao diện web** theo kiểu hiện đại và dễ dùng hơn input file thường. Nó hỗ trợ sẵn nhiều thứ như:

- chọn file bằng giao diện đẹp
- xem trước ảnh
- giới hạn loại file
- xóa file đã chọn
- quản lý danh sách file bằng state

Trong phần này, FilePond được dùng để xử lý **avatar**

- Code:

```
registerPlugin(
  FilePondPluginFileValidateType,
  FilePondPluginImagePreview
);
```

Đoạn này dùng để **gắn thêm chức năng mở rộng** cho FilePond :

- **FilePondPluginFileValidateType:** Plugin này giúp kiểm tra **loại file** mà người dùng chọn.
- **FilePondPluginImagePreview:** Plugin này giúp **xem trước ảnh** ngay trên giao diện sau khi chọn file.

```
useEffect(() => {
  if(infoUser) {
    // Hiển avatar mặc định
    if(infoUser.avatar) {
      setAvatars([
        {
          source: infoUser.avatar
        }
      ]);
    }
```

Ý nghĩa:

- nếu user đã có avatar cũ trong database
- thì đưa avatar đó vào state avatars
- rồi truyền state này xuống FilePond
- kết quả là FilePond hiển thị ảnh cũ ra trước

```
  validator
    .addField('#fullName', [
      {
        rule: 'required',
        errorMessage: 'Vui lòng nhập họ tên!'
      },
      {
        rule: 'minLength',
        value: 5,
        errorMessage: 'Họ tên phải có ít nhất 5 ký tự!',
      },
      {
        rule: 'maxLength',
        value: 50,
        errorMessage: 'Họ tên không được vượt quá 50 ký tự!',
      },
    ])
    .addField('#email', [
      {
        rule: 'required',
        errorMessage: 'Vui lòng nhập email của bạn!',
      },
      {
        rule: 'email',
        errorMessage: 'Email không đúng định dạng!',
      },
    ])
```

Đây là phần code để validate dữ liệu nhập vào form thông tin cá nhân:

const validator = new JustValidate("#profileForm");

Dòng này tạo bộ kiểm tra dữ liệu cho form có id là profileForm.

Sau đó:

.addField('#fullName', [...])

nghĩa là thêm luật kiểm tra cho ô fullName.

Cụ thể:

- required: bắt buộc nhập
- minLength: 5: ít nhất 5 ký tự
- maxLength: 50: không quá 50 ký tự

Tiếp theo:

.addField('#email', [...])

kiểm tra cho ô email:

- bắt buộc nhập
- phải đúng định dạng email

Vai trò của đoạn này trong luồng hoạt động

Khi người dùng nhập sai:

- JustValidate sẽ hiện lỗi ngay ở giao diện
- ngăn người dùng nhập dữ liệu không hợp lệ

```
const handleSubmit = (event: any) => {
  event.preventDefault();
  const fullName = event.target.fullName.value;
  const email = event.target.email.value;
  const phone = event.target.phone.value;
  let avatar = null;
  if(avatars.length > 0) {
    avatar = avatars[0].file;
  }

  // Tạo FormData
  const formData = new FormData();
  formData.append("fullName", fullName);
  formData.append("email", email);
  formData.append("phone", phone);
  formData.append("avatar", avatar);

  fetch(`${process.env.NEXT_PUBLIC_API_URL}/user/profile`, {
    method: "PATCH",
    body: formData,
    credentials: "include", // Gửi kèm cookie
  })
    .then(res => res.json())
    .then(data => {
      if(data.code == "error") {
        toast.error(data.message);
      }

      if(data.code == "success") {
        toast.success(data.message);
      }
    })
}
```

Đây là hàm handleSubmit trong đó:

event.preventDefault();

Mặc định form HTML khi submit sẽ reload lại trang.
Dòng này chặn hành vi đó để xử lý bằng JavaScript.

const fullName = event.target.fullName.value;
const email = event.target.email.value;
const phone = event.target.phone.value;

Đoạn code này dùng để lấy dữ liệu từ form

let avatar = null;
if(avatars.length > 0) {

```
  avatar = avatars[0].file;
}
```

Ý nghĩa:

- mặc định chưa có file avatar mới thì avatar = null
- nếu có file trong avatars thì lấy file đầu tiên

Vì ở FilePond bạn chỉ cho chọn 1 file:

allowMultiple={false}

nên chỉ cần lấy phần tử đầu tiên.

```
const formData = new FormData();
formData.append("fullName", fullName);
formData.append("email", email);
formData.append("phone", phone);
formData.append("avatar", avatar);
```

Dùng FormData để gửi dữ liệu vì đang gửi cả:

- dữ liệu text
- file ảnh

Nếu chỉ gửi JSON thì không phù hợp để upload file.

FormData sẽ đóng gói dữ liệu theo dạng multipart/form-data, rất hay dùng khi upload avatar, ảnh sản phẩm, file đính kèm.

```
fetch(`${process.env.NEXT_PUBLIC_API_URL}/user/profile`, {
  method: "PATCH",
  body: formData,
  credentials: "include",
})
```

Ý nghĩa:

- gửi request tới API cập nhật profile
- method là PATCH vì đang cập nhật thông tin
- body là formData
- credentials: "include" nghĩa là gửi kèm cookie đăng nhập

Cookie này thường dùng để backend biết người dùng nào đang đăng nhập.

```
.then(res => res.json())
.then(data => {
```

```
  if(data.code == "error") {
    toast.error(data.message);
  }

  if(data.code == "success") {
    toast.success(data.message);
  }
})
```

Sau khi backend trả về:

- nếu lỗi thì hiện thông báo đỏ
- nếu thành công thì hiện thông báo xanh

toast ở đây là thông báo nổi trên màn hình.

```
  <FilePond
    name="avatar"
    allowMultiple={false} // Chỉ chọn 1 ảnh
    allowRemove={true} // Cho phép xóa ảnh
    labelIdle='+'
    acceptedFileTypes={['image/*']}
    onupdatefiles={setAvatars}
    files={avatars}
  />
</div>
```

Đây là nơi hiển thị component upload avatar.

name="avatar"

Tên trường file.
Nó tương ứng với ý nghĩa đây là file avatar.

2.1.2 Xử lý bên phía Backend

```
router.patch(
  "/profile",
  authMiddleware.verifyTokenUser,
  upload.single("avatar"),
  userController.profilePatch
)
```

Định nghĩa ra route có: đường dẫn là “/profile”

```
export const profilePatch = async (req: AccountRequest, res: Response) => {
  if(req.file) {
    req.body.avatar = req.file.path;
  } else {
    delete req.body.avatar;
  }

  await AccountUser.updateOne({
    _id: req.account.id
  }, req.body);

  res.json({
    code: "success",
    message: "Cập nhật thành công!"
  })
}
```

Tạo Controller profilePatch:

Trong controller này, dữ liệu được ghi vào MongoDB ở lệnh await AccountUser.updateOne(...). Trước đó hệ thống kiểm tra nếu có upload avatar mới thì lấy đường dẫn ảnh từ req.file.path gán vào req.body.avatar. Sau đó dùng updateOne để cập nhật document người dùng theo _id của tài khoản đang đăng nhập. Cuối cùng backend trả JSON thành công về cho frontend.

```
import { v2 as cloudinary } from "cloudinary";
import { CloudinaryStorage } from 'multer-storage-cloudinary';

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET
});

export const storage = new CloudinaryStorage({
  cloudinary: cloudinary
});
```

- Giới thiệu thư viện

cloudinary

Đây là SDK của Cloudinary cho JavaScript/Node.js.
Nó dùng để kết nối ứng dụng của bạn với dịch vụ Cloudinary, từ đó có thể:

- upload ảnh
- lưu ảnh trên cloud
- lấy URL ảnh
- xóa/sửa ảnh nếu cần

Nói ngắn gọn: **thư viện này là cầu nối giữa code backend và Cloudinary**.

multer-storage-cloudinary

Đây là thư viện tích hợp giữa:

- multer: middleware xử lý file upload từ form/request
- cloudinary: nơi lưu file

Bình thường multer thường lưu file vào thư mục local.
Còn khi dùng multer-storage-cloudinary, file upload sẽ được **đưa thẳng lên Cloudinary**, không cần lưu tạm ở máy chủ.

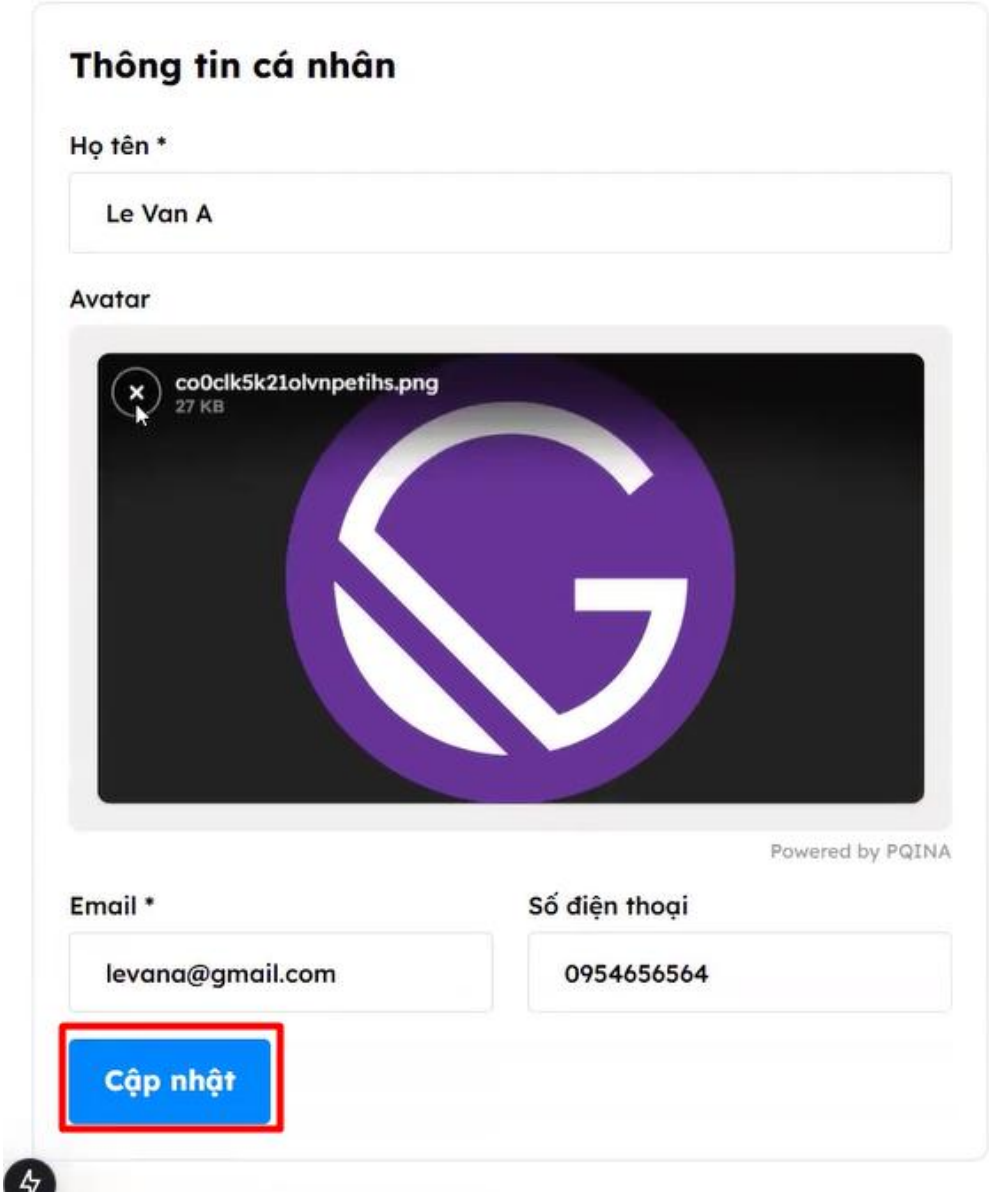
=> Đoạn code này dùng thư viện cloudinary để kết nối backend với dịch vụ lưu ảnh Cloudinary, và dùng multer-storage-cloudinary để tích hợp Cloudinary với multer. Sau khi cấu hình cloud_name, api_key, api_secret bằng biến môi trường, em tạo một storage kiểu CloudinaryStorage. Storage này sẽ được truyền cho multer, để khi người dùng upload file thì file được lưu trực tiếp lên Cloudinary thay vì lưu trong máy chủ

```
export const verifyTokenUser = async (
  req: AccountRequest,
  res: Response,
  next: NextFunction
) => {
  try {
    const token = req.cookies.token;

    if(!token) {
      res.json({
        code: "error",
        message: "Vui lòng gửi kèm theo token!"
      })
      return;
    }

    var decoded = jwt.verify(token, `${process.env.JWT_SECRET}`) as jwt.JwtPayload;
    const { id, email } =decoded;

    const existAccount = await AccountUser.findOne({
      _id: id,
      email: email,
    })

    if(!existAccount) {
      res.clearCookie("token");
      res.json({
        code: "error",
        message: "Token không hợp lệ!"
      })
      return;
    }

    req.account = existAccount;

    next();
  } catch (error) {
    res.clearCookie("token");
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}
```

Đoạn code này là **middleware xác thực đăng nhập bằng JWT**.
Mục đích của nó là:

- lấy token từ cookie
- kiểm tra token có hợp lệ không
- giải mã token để lấy id, email
- kiểm tra tài khoản đó có tồn tại trong database không
- nếu hợp lệ thì cho đi tiếp tới controller bằng next()

2.1.3. Kết quả



```
_id: ObjectId('68ef9ca2a7f69f998edbd14d')
fullName : "Le Van A"
email : "levana@gmail.com"
password : "$2b$10$QqEid1QBowAPzGxiEwqfluN7MFuMrHZUkF4chgTN5s9Jjr2vBsTLK"
createdAt : 2025-10-15T13:07:46.717+00:00
updatedAt : 2025-10-22T12:54:56.995+00:00
__v : 0
avatar : "https://res.cloudinary.com/dxhwpfxxn/image/upload/v1761137703/co0clk5k…"
phone : "0954656564"
```

Sau khi ấn cập nhật dữ liệu được lưu vào CSDL online và hiện thị mặc định trên giao diện

2.2. Trang thông tin công ty (nhà tuyển dụng)

2.2.1. Xử lý phía Frontend

```
// Register the plugin
registerPlugin(
  FilePondPluginFileValidateType,
  FilePondPluginImagePreview
);
```

Đoạn này dùng để **gắn thêm chức năng mở rộng** cho FilePond :

- **FilePondPluginFileValidateType:** Plugin này giúp kiểm tra **loại file** mà người dùng chọn.
- **FilePondPluginImagePreview:** Plugin này giúp **xem trước ảnh** ngay trên giao diện sau khi chọn file.

```
useEffect(() => {
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/city/list`)
    .then(res => res.json())
    .then(data => {
      if(data.code == "success") {
        setCityList(data.cityList);
      }
    })
}, []);
```

Đoạn code này dùng useEffect để gọi API lấy danh sách thành phố khi component được render lần đầu. Sau khi gọi fetch tới endpoint /city/list, dữ liệu trả về được chuyển sang JSON. Nếu backend trả về thành công thì em cập nhật state cityList bằng setCityList, từ đó giao diện có thể hiển thị danh sách tỉnh thành cho người dùng chọn.

```
useEffect(() => {
  if(infoCompany) {
    // Hiển logo mặc định
    if(infoCompany.logo) {
      setLogos([
        {
          source: infoCompany.logo
        }
      ]);
    }
```

Đoạn này dùng useEffect để khi dữ liệu công ty được load về thì kiểm tra xem công ty đã có logo chưa. Nếu có, em đưa logo đó vào state logos dưới dạng source. State này sẽ được truyền vào component upload để hiển thị logo mặc định trên form.

```
    // Validate dữ liệu
    const validator = new JustValidate("#profileForm");

    validator
      .addField('#companyName', [
        {
          rule: 'required',
          errorMessage: 'Vui lòng nhập tên công ty!'
        },
        {
          rule: 'maxLength',
          value: 200,
          errorMessage: 'Tên công ty không được vượt quá 200 ký tự!',
        },
      ])
      .addField('#email', [
        {
          rule: 'required',
          errorMessage: 'Vui lòng nhập email của bạn!',
        },
        {
          rule: 'email',
          errorMessage: 'Email không đúng định dạng!',
        },
      ])
  }
}, [infoCompany]);
```

Đoạn code này dùng JustValidate để kiểm tra dữ liệu form profile. Em tạo validator cho form profileForm, sau đó thêm rule cho trường companyName gồm bắt buộc nhập và tối đa 200 ký tự, và trường email gồm bắt buộc nhập và đúng định dạng email. Khi người dùng nhập sai, thư viện sẽ hiển thị thông báo lỗi ngay trên giao diện.

```
const handleSubmit = (event: any) => {
  event.preventDefault();
  const companyName = event.target.companyName.value;
  const address = event.target.address.value;
  const companyModel = event.target.companyModel.value;
  const companyEmployees = event.target.companyEmployees.value;
  const workingTime = event.target.workingTime.value;
  const workOvertime = event.target.workOvertime.value;
  const email = event.target.email.value;
  const phone = event.target.phone.value;
  const description = event.target.description.value;
  const city = event.target.city.value;
  let logo = null;
  if(logos.length > 0) {
    logo = logos[0].file;
  }

  // Tạo FormData
  const formData = new FormData();
  formData.append("companyName", companyName);
  formData.append("address", address);
  formData.append("companyModel", companyModel);
  formData.append("companyEmployees", companyEmployees);
  formData.append("workingTime", workingTime);
  formData.append("workOvertime", workOvertime);
  formData.append("email", email);
  formData.append("phone", phone);
  formData.append("description", description);
  formData.append("logo", logo);
  formData.append("city", city);

  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/profile`, {
    method: "PATCH",
    body: formData,
    credentials: "include", // Gửi kèm cookie
  })
    .then(res => res.json())
    .then(data => {
      if(data.code == "error") {
        toast.error(data.message);
      }

      if(data.code == "success") {
        toast.success(data.message);
      }
    })
}
```

Đoạn này có tác dụng **lấy dữ liệu người dùng nhập trong form, kèm logo nếu có, rồi gửi sang backend để cập nhật hồ sơ công ty**.

Luồng hoạt động:

- event.preventDefault() để chặn form reload trang.

- Lấy các giá trị từ form như: tên công ty, địa chỉ, email, điện thoại, giờ làm, mô tả, thành phố...
- Nếu có logo trong logos thì lấy file logo đó ra.
- Tạo FormData để gói cả **dữ liệu text** và **file logo**.
- Gửi request PATCH tới API /company/profile.
- credentials: "include" để gửi kèm cookie đăng nhập.
- Nếu backend trả lỗi thì hiện toast.error, nếu thành công thì hiện toast.success.

```
  <FilePond
    name="logo"
    allowMultiple={false} // Chỉ chọn 1 ảnh
    allowRemove={true} // Cho phép xóa ảnh
    labelIdle='+'
    acceptedFileTypes={['image/*']}
    onupdatefiles={setLogos}
    files={logos}
  />
</div>
```

đây là component chọn logo, chỉ nhận 1 ảnh, có thể xóa, và đồng bộ dữ liệu ảnh với state logos.

2.2.2 Xử lý bên phía Backend

```
import { Router } from "express";
import * as cityController from "../controllers/city.controller";

const router = Router();

router.get("/list", cityController.list)

export default router;
```

city route: khai báo API lấy danh sách thành phố

```
import { Router } from "express";
import * as companyController from "../controllers/company.controller";
import * as companyValidate from "../validates/company.validate";
import multer from "multer";
import { storage } from "../helpers/cloudinary.helper";
import * as authMiddleware from "../middlewares/auth.middleware";

const router = Router();

const upload = multer({ storage: storage });

router.post(
  "/register",
  companyValidate.registerPost,
@@ -16,4 +21,11 @@ router.post(
  companyController.loginPost
)

router.patch(
  "/profile",
  authMiddleware.verifyTokenCompany,
  upload.single("logo"),
  companyController.profilePatch
)

export default router;
```

company route: khai báo các API của công ty

- verifyTokenCompany: kiểm tra đăng nhập
- upload.single("logo"): nhận 1 ảnh logo
- profilePatch: cập nhật thông tin công ty vào database

```
import { Request, Response } from "express";
import City from "../models/city.model";

export const list = async (req: Request, res: Response) => {
  const cityList = await City.find({});

  res.json({
    code: "success",
    message: "Thành công!",
    cityList: cityList
  });
}
```

city.controller

Đây là hàm **lấy toàn bộ danh sách thành phố từ MongoDB** rồi trả về cho frontend.

- City.find({}): truy vấn tất cả bản ghi trong collection city

- res.json(...): trả dữ liệu về dưới dạng JSON
- cityList: cityList: frontend sẽ nhận mảng thành phố này để đổ ra select/dropdown

```
import AccountCompany from "../models/account-company.model";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { AccountRequest } from "../interfaces/request.interface";

export const registerPost = async (req: Request, res: Response) => {
  const existAccount = await AccountCompany.findOne({
@@ -77,3 +78,20 @@ export const loginPost = async (req: Request, res: Response) => {
    message: "Đăng nhập thành công!"
  })
}

export const profilePatch = async (req: AccountRequest, res: Response) => {
  if(req.file) {
    req.body.logo = req.file.path;
  } else {
    delete req.body.logo;
  }

  await AccountCompany.updateOne({
    _id: req.account.id
  }, req.body);

  res.json({
    code: "success",
    message: "Cập nhật thành công!"
  })
}
\ No newline at end of file
```

company.controller

Đây là hàm **cập nhật hồ sơ công ty**.

Luồng chính:

- Nếu có upload logo mới:
  - req.file.path là đường dẫn logo sau khi upload
  - gán vào req.body.logo
- Nếu không có logo mới:
  - delete req.body.logo
  - để không ghi đè logo cũ
- AccountCompany.updateOne(...):
  - tìm công ty theo req.account.id

    - cập nhật dữ liệu mới trong req.body vào MongoDB
- res.json(...):
    - báo frontend biết cập nhật thành công

```
export const verifyTokenCompany = async (
  req: AccountRequest,
  res: Response,
  next: NextFunction
) => {
  try {
    const token = req.cookies.token;

    if(!token) {
      res.json({
        code: "error",
        message: "Vui lòng gửi kèm theo token!"
      })
      return;
    }

    var decoded = jwt.verify(token, `${process.env.JWT_SECRET}`) as jwt.JwtPayload;
    const { id, email } =decoded;

    const existAccount = await AccountCompany.findOne({
      _id: id,
      email: email,
    })

    if(!existAccount) {
      res.clearCookie("token");
      res.json({
        code: "error",
        message: "Token không hợp lệ!"
      })
      return;
    }

    req.account = existAccount;

    next();
  } catch (error) {
    res.clearCookie("token");
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}
```

Đoạn này là **middleware kiểm tra đăng nhập của công ty bằng JWT token**.

Nó dùng để:

- lấy token từ cookie
- kiểm tra token có hợp lệ không
- giải mã token để lấy id, email
- tìm công ty tương ứng trong MongoDB
- nếu hợp lệ thì gắn thông tin tài khoản vào req.account
- rồi cho request đi tiếp bằng next()

2.2.3. Kết quả

```
_id: ObjectId('68f23eb24e45470e1672f80d')
companyName : "ABC.LTD"
email : "contact@abc.com"
password : "$2b$10$O0FSRgB5baHiAl/UNcOCGeD9Wj3yPoLtewir/txGYeglMveg.NEJO"
createdAt : 2025-10-17T13:03:46.162+00:00
updatedAt : 2025-10-22T13:29:47.496+00:00
__v : 0
address : "Số 123, đường ABC"
companyEmployees : "500+ nhân viên"
companyModel : "Sản phẩm"
description : "Mô tả chi tiết..."
logo : "https://res.cloudinary.com/dxhwpfxxn/image/upload/v1761139793/v69haeob…"
phone : "19001234"
workOvertime : "Không có OT"
workingTime : "Thứ 2 - Thứ 6"
```


Sau khi ấn cập nhật dữ liệu được lưu vào CSDL online và hiện thị mặc định trên giao diện

3. Kết quả đạt được

• Xây dựng thành công giao diện thông tin cá nhân cho ứng viên, hiển thị được dữ liệu mặc định từ hệ thống.
• Xây dựng thành công giao diện thông tin công ty cho nhà tuyển dụng, hỗ trợ chỉnh sửa các thông tin cơ bản của công ty.
• Triển khai validate dữ liệu form ở phía frontend bằng JustValidate cho các trường quan trọng như họ tên, tên công ty và email.
• Tích hợp thành công FilePond để upload avatar và logo, hỗ trợ xem trước ảnh và chỉ chấp nhận file ảnh hợp lệ.
• Kết nối Cloudinary để lưu trữ ảnh upload từ người dùng thay vì lưu trực tiếp trên server.
• Xây dựng API lấy danh sách thành phố từ MongoDB và trả dữ liệu về frontend để hiển thị trong form.
• Xây dựng thành công API cập nhật thông tin cá nhân và API cập nhật thông tin công ty bằng phương thức PATCH.
• Triển khai middleware xác thực token cho user và company, giúp bảo vệ các API cập nhật dữ liệu.
• Cập nhật thành công dữ liệu hồ sơ từ frontend lên MongoDB, bao gồm cả trường hợp có thay đổi avatar hoặc logo.

4. Khó khăn gặp phải

• Gặp khó khăn trong việc hiển thị avatar và logo mặc định lên form khi dữ liệu được load từ backend về.
• Ban đầu chưa phân biệt rõ giữa dữ liệu ảnh cũ hiển thị trên giao diện và file ảnh mới người dùng chọn để upload.

• Việc gửi đồng thời dữ liệu text và file ảnh lên backend khiến em phải tìm hiểu cách dùng FormData thay cho JSON thông thường.
• Trong quá trình cập nhật hồ sơ, em gặp vướng mắc khi xử lý trường hợp người dùng không chọn ảnh mới nhưng vẫn bấm cập nhật, vì nếu xử lý không đúng có thể làm mất ảnh cũ.
• Em gặp khó khăn khi cấu hình upload ảnh lên Cloudinary và kết nối Cloudinary với multer.
• Việc xác thực token bằng cookie ở các API cập nhật hồ sơ cũng gây nhầm lẫn ở giai đoạn đầu, đặc biệt là luồng lấy token từ cookie, giải mã token và gắn thông tin tài khoản vào req.account.
• Cần thêm thời gian để tổ chức luồng xử lý frontend – middleware – controller – model sao cho rõ ràng, tránh nhầm lẫn giữa phần hiển thị dữ liệu và phần cập nhật dữ liệu.

5. Hướng xử lý

• Đối với việc hiển thị ảnh mặc định: em sử dụng useEffect để kiểm tra dữ liệu trả về từ backend, sau đó gán avatar hoặc logo cũ vào state dưới dạng source để FilePond hiển thị lại trên form.
• Đối với validate dữ liệu: em dùng thư viện JustValidate để kiểm tra các trường bắt buộc, độ dài dữ liệu và định dạng email trước khi submit form.
• Đối với việc gửi dữ liệu có kèm ảnh: em sử dụng FormData để đóng gói toàn bộ dữ liệu text và file ảnh rồi gửi lên backend bằng fetch.
• Đối với upload ảnh: em cấu hình multer kết hợp với multer-storage-cloudinary, từ đó đưa file ảnh lên Cloudinary và lấy path của ảnh để lưu vào database.
• Đối với xác thực người dùng: em xây dựng middleware kiểm tra token trong cookie, dùng JWT để giải mã token, sau đó truy vấn database để xác nhận tài khoản tồn tại trước khi cho phép cập nhật dữ liệu.
• Đối với cập nhật hồ sơ: em sử dụng updateOne() để cập nhật dữ liệu vào MongoDB; nếu có file ảnh mới thì gán thêm đường dẫn ảnh vào req.body, còn nếu không có ảnh mới thì xóa trường ảnh khỏi req.body để tránh ghi đè dữ liệu cũ.
• Đối với danh sách thành phố: em xây dựng API riêng để lấy dữ liệu từ collection city, sau đó gọi từ frontend bằng useEffect và lưu vào state để hiển thị.

6. Kế hoạch tuần tiếp theo

Trong tuần tiếp theo, em dự kiến tiếp tục phát triển các chức năng liên quan đến quản lý công việc trên hệ thống website tuyển dụng.

Cụ thể:

• Xây dựng trang thêm công việc cho nhà tuyển dụng, cho phép nhập các thông tin như tiêu đề công việc, mô tả, mức lương, địa điểm, yêu cầu ứng viên, v.v.

• Thiết kế và hoàn thiện giao diện form thêm công việc, đảm bảo dễ sử dụng và hiển thị đầy đủ các trường thông tin cần thiết.

• Triển khai validate dữ liệu cho form thêm công việc nhằm đảm bảo dữ liệu nhập vào hợp lệ trước khi gửi lên backend.

• Xây dựng API backend để lưu thông tin công việc vào cơ sở dữ liệu MongoDB.

• Xây dựng trang quản lý công việc cho nhà tuyển dụng, cho phép hiển thị danh sách các công việc đã đăng.

• Triển khai các chức năng chỉnh sửa và xóa công việc trên trang quản lý.

• Kết nối dữ liệu giữa frontend và backend để hiển thị danh sách công việc theo tài khoản công ty.

• Kiểm tra và tối ưu luồng xử lý dữ liệu giữa frontend – backend – database để đảm bảo hệ thống hoạt động ổn định.